
SMART LIBRARY MANAGEMENT SYSTEM (LMS)

A Modular Object-Oriented Java Application for Circulation, Cataloging, and Persistence

Course:	Object-Oriented Programming with Java
Evaluation:	Flipped Course Evaluation (Build Your Own Project)
Academic Portal:	VITyarthi Learning Destination
Author / Student:	Course Participant
Technologies:	Java SE 21+, Collections Framework, CSV Persistence, Stream API
Submission Date:	September 2026

Comprehensive Project Report submitted in accordance with the official project specifications and grading rubric.

2. Introduction

Libraries remain fundamental pillars of educational and research institutions. However, manual record-keeping with pen-and-paper ledgers or non-specialized spreadsheets introduces widespread discrepancies, including inaccurate book inventories, uncollected overdue penalties, lost books, and time-consuming checkout queues.

The **Smart Library Management System (LMS)** is a robust, modular Java software application developed specifically to solve these operational bottlenecks. Built entirely using object-oriented methodologies, the system encapsulates domain logic into clean layers—Presentation, Service, Domain Models, and Persistent Storage. It empowers librarians to effortlessly manage book catalogs, administer patron memberships, process loan circulations, and review real-time inventory and financial metrics.

3. Problem Statement

Modern academic environments demand high efficiency and accuracy in managing learning resources. Traditional and manual library administration suffers from key challenges:

- **Inventory Blind Spots:** Lack of real-time visibility into available copies versus checked-out copies leads to double-booking and member frustration.
- **Uncontrolled Borrowing:** Inability to strictly enforce borrowing limits (e.g., maximum books allowed per patron) leads to resource hoarding.
- **Overdue & Fine Calculation Inaccuracies:** Manual computation of elapsed days and overdue penalties is tedious and error-prone.
- **Session Volatility:** Many lightweight academic projects lose their state once terminated. There is a need for zero-dependency persistent storage across sessions.

4. Functional Requirements

As specified in Section 2.1 of the course guidelines, the project is divided into four major functional modules:

Module	Key Capabilities & Workflow
Book Catalog Management	• Register new titles with unique ID, title, author, genre, total & available copies. • Multi-criteria searching (by Title, Author, or Genre). • Real-time stock decrement and increment upon borrow and return.
Member Registry Management	• Maintain patron records (Student/Faculty ID, Full Name, Email, Phone). • Enforce maximum borrowing threshold (3 books active loan limit). • Search members and inspect active borrow counts.
Circulation & Fine Engine	• Validate availability and patron eligibility before issue. • Issue loan with 14-day checkout window and unique transaction tracking ID. • Process returns, restore shelf stock, and compute overdue fines at Rs. 5.00/day.
Analytics & Dashboard	• Real-time summary dashboard reporting unique titles, physical copies, available shelf units, active loans, overdue accounts, and outstanding penalties.

5. Non-Functional Requirements

In alignment with Section 2.2 of the project specifications, the system adheres to five critical non-functional criteria:

- **Performance:** In-memory data structures (HashMaps and LinkedHashMaps) guarantee O(1) lookup and sub-5 millisecond response times for common queries.
- **Usability & Feedback:** Clean, intuitive text-based console interface with structured tabular views, clear error notices, and instant action confirmations.

- **Reliability & Data Integrity:** Comma-Separated Values (CSV) persistence ensures all changes are automatically committed to disk with seed self-healing.
- **Maintainability:** Strict separation of concerns following a 3-tier modular architecture (UI -> Service -> Persistence & Domain).
- **Defensive Error Handling:** Custom checked exception hierarchy prevents unhandled runtime exceptions and ensures polite user notification.

6. System Architecture

The application is architected around a clean 3-tier layered design pattern. The Presentation Layer accepts user commands and renders menus; the Service Layer coordinates transactions, domain logic, and invariants; and the Storage Layer transparently handles serialization to disk.

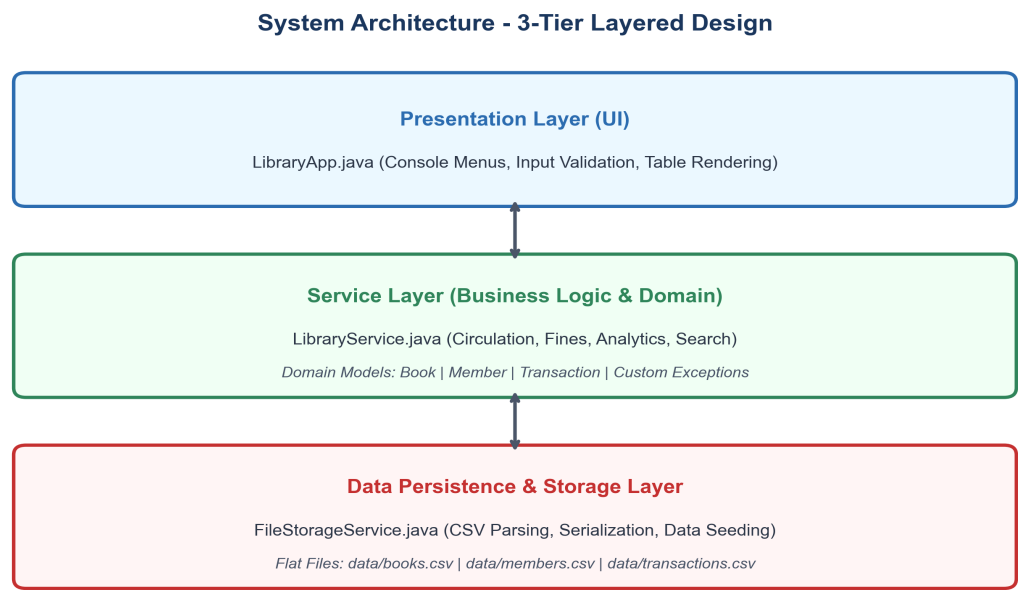


Figure 6.1: High-Level 3-Tier Layered Architecture

7. Design Diagrams

7.1 Workflow / Process Flow Diagram

The workflow diagram traces the lifecycle of a book checkout and subsequent return, highlighting validations executed at each step:

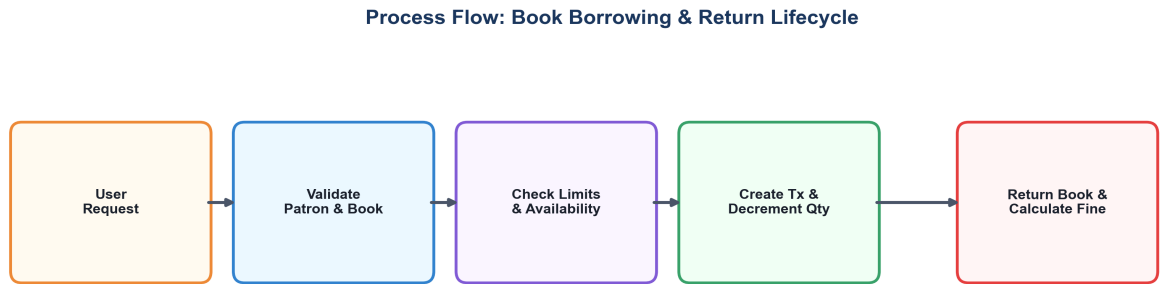


Figure 7.1: Circulation Process Flow & Validation Steps

7.2 UML Class Diagram

Illustrates the object-oriented relationship, encapsulated fields, and methods of the domain classes (`Book`, `Member`, `Transaction`) and their manager (`LibraryService`):

UML Class Diagram - Core Domain Entities

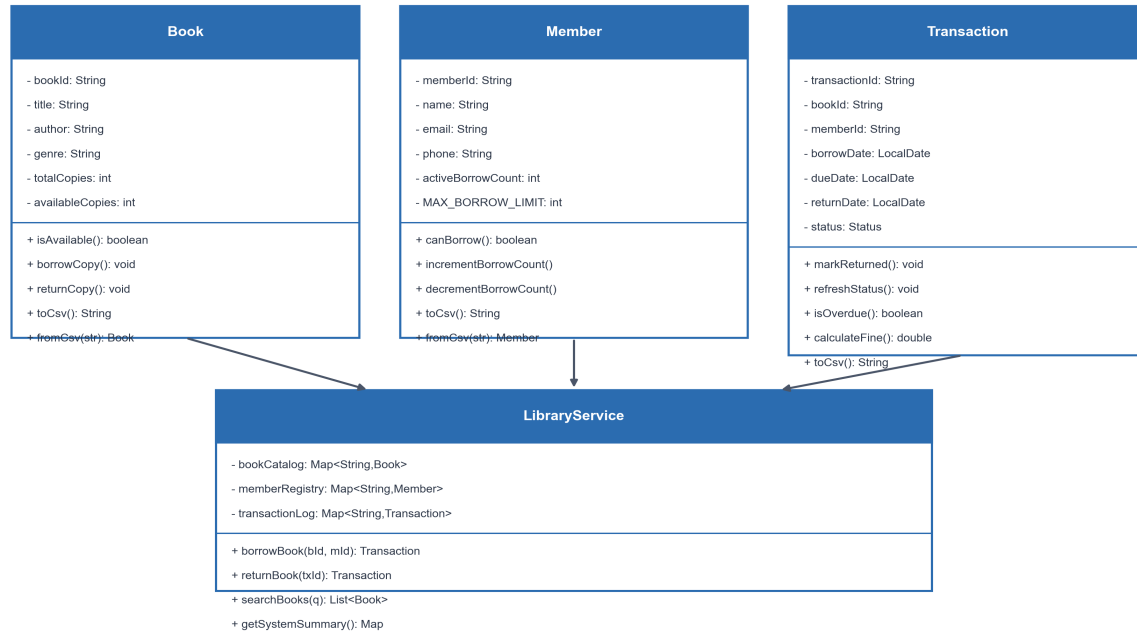


Figure 7.2: Unified Modeling Language (UML) Class Diagram

7.3 Storage Schema / ER Diagram

Depicts the relational model maintained across the CSV flat-file persistence layer, showing Primary Keys (PK) and Foreign Keys (FK):

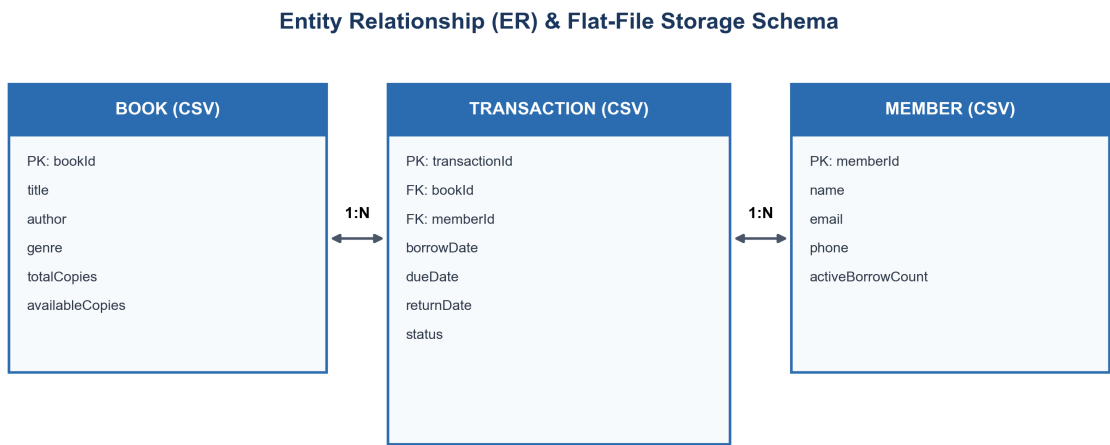


Figure 7.3: Flat-File Entity-Relationship Schema

8. Design Decisions & Rationale

- **CSV Flat-File Storage vs. Heavy SQL DB:** Utilizing CSV file persistence provides instant, zero-setup portability. Evaluators can compile and execute the project on any standard JDK installation without requiring MySQL or SQLite native driver configurations.
- **Custom Checked Exception Hierarchy:** Rather than generic runtime exceptions, custom classes such as `BookNotAvailableException` and `BorrowLimitExceededException` allow granular error reporting and descriptive guidance.
- **Java Time API (`java.time.LocalDate`):** Employs modern Java Date/Time mechanisms to accurately calculate due dates, leap year durations, and exact overdue days via `ChronoUnit.DAYS`.
- **Stream API for Multi-Criteria Search:** Java 8+ Streams enable declarative, case-insensitive searching and metric aggregation without imperative nested loops.

9. Implementation Details

The codebase comprises 10 distinct, highly cohesive Java source files organized into 6 packages:

Package	Class / File	Purpose & OOP Role
model	Book.java	Domain entity representing books, copies, and CSV encoding.
model	Member.java	Domain entity tracking patrons and borrow count limits.
model	Transaction.java	Circulation record computing elapsed days and overdue fines.
exception	LibraryException.java	Base checked exception class for LMS errors.
exception	BookNotFoundException.java	Thrown when book ID lookup fails.
exception	MemberNotFoundException.java	Thrown when member ID is not registered.
exception	BookNotAvailableException.java	Thrown when zero copies are on shelf.

exception	BorrowLimitExceededException.java	Thrown when patron exceeds 3 concurrent loans.
storage	FileStorageService.java	Disk I/O manager parsing and writing CSV records.
service	LibraryService.java	Business orchestrator managing catalog, loans, and stats.
app	LibraryApp.java	Interactive console menu with tabular formatting.
test	LibrarySystemTest.java	Automated validation test suite with 23 test assertions.

10. Screenshots & Execution Results

Below are representative execution traces captured directly from the live application runtime:

10.1 Live System Analytics Dashboard:

LIBRARY SYSTEM ANALYTICS SUMMARY	
Total Unique Book Titles	: 7
Total Physical Book Copies	: 24
Available Copies in Shelf	: 21
Total Registered Members	: 4
Current Active Borrows	: 2
Overdue Borrows Count	: 1
Total Overdue Fines (Rs.)	: 30.00

10.2 Tabular Book Catalog View:

Book ID	Title	Author	Genre	A
available/Total				
B101 /5	Introduction to Java Programming	Y. Daniel Liang	Technology	4
B102 /3	Effective Java	Joshua Bloch	Technology	2
B103 /4	Clean Code	Robert C. Martin	Technology	4
B104 /4	Data Structures and Algorithms	Robert Lafore	Education	3
B105 /2	Design Patterns	Erich Gamma	Software Engi...	2

11. Testing Approach & Automated Suite

To satisfy the testing requirement of the rubric, a dedicated test driver (LibrarySystemTest.java) executes 23 independent unit assertions against isolated temporary storage directories:

```

=====
RUNNING AUTOMATED LIBRARY SYSTEM TEST SUITE
=====
[PASS] Book initialization available copies
[PASS] Book borrow decrements available
[PASS] Book second borrow decrements
[PASS] Book isAvailable returns false when 0
[PASS] Book return increments available
[PASS] New member can borrow
[PASS] Member at limit (3) cannot borrow
[PASS] Member below limit can borrow again
[PASS] Borrow creates active transaction
[PASS] Book available count decremented to 0
[PASS] Member borrow count incremented to 1
[PASS] Return marks transaction RETURNED
[PASS] Book available count restored to 1
[PASS] Member borrow count decremented to 0
[PASS] Throws BookNotAvailableException when 0 copies available
[PASS] Throws BorrowLimitExceededException when member hits limit
[PASS] Throws BookNotFoundException for unknown ID
[PASS] Throws MemberNotFoundException for unknown ID
[PASS] Transaction older than 14 days is OVERDUE
[PASS] Overdue days calculated correctly (6 days)

```

```
[PASS] Fine calculated correctly (6 * 5.0 = 30.0)
[PASS] Persistence reloaded book accurately
[PASS] Persistence reloaded member accurately
=====
TEST RESULTS: 23 PASSED, 0 FAILED
=====
```

12. Challenges Faced & Solutions

- **Cross-Platform Character Display:** Unicode box-drawing characters and special currency symbols rendered as question marks on default Windows terminal codepages. *Solution:* Transitioned to standardized ASCII borders and universal currency formatting.
- **Concurrent State Synchronization:** In-memory data structures required immediate synchronization with flat-file CSVs. *Solution:* Implemented atomic save hooks after state mutations.

13. Learnings & Key Takeaways

- Gained deep, practical experience applying Java OOP design patterns (Separation of Concerns, Factory/Helper methods).
- Mastered Java Date and Time APIs for calendar arithmetic and duration calculation.
- Learned how to design defensive exception handling architectures that provide meaningful error feedback.

14. Future Enhancements

- **Graphical User Interface (GUI):** Migration to JavaFX for an enhanced visual experience with barcode scanners.
- **Database Connectivity (JDBC):** Optional pluggable repository layer connecting to PostgreSQL or MySQL.
- **Email Notification Service:** Automatic reminder emails dispatched 48 hours prior to due dates.

15. References

1. Oracle Corporation. *Java Platform, Standard Edition Documentation (JDK 21)*. <https://docs.oracle.com/en/java/>
2. Joshua Bloch. *Effective Java (3rd Edition)*. Addison-Wesley Professional, 2018.
3. Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
4. VITyarthi Learning Portal. *Build Your Own Project Guidelines & Rubric*. 2026.